Reflection: FastAPI + PostgreSQL + pgAdmin Project

Sandra Lekhana

1. Objective

The goal of this project was to build and containerize a complete backend environment using **FastAPI**, **PostgreSQL**, and **pgAdmin** via **Docker Compose**.
The exercise focused on understanding how to orchestrate multiple interconnected services, perform SQL operations, and manage a reproducible development environment for data-driven applications.

---

2. Approach

I began by creating a structured project folder containing separate directories for the FastAPI application (/app) and database initialization scripts (/db).
I configured a docker-compose.yml file to run three services:

- fastapi_app for the API server,
- fastapi_db for PostgreSQL, and
- pgadmin for database administration.

Environment variables such as database credentials and port mappings were managed through a .env file for security and portability.
Once the containers were running, I connected pgAdmin to the PostgreSQL service using the internal hostname db, verified the connection, and executed SQL commands for table creation, data insertion, querying, updating, and deletion.

---

3. Key Learnings

- **Containerization:** I learned how to use Docker Compose to define, build, and run multiple containers that communicate seamlessly within a shared network.
- **Database Integration:** Configuring PostgreSQL and linking it to FastAPI helped me understand persistent storage, schema creation, and one-to-many relationships.

- **pgAdmin Usage:** I gained confidence using pgAdmin as a database management GUI for executing SQL queries and inspecting schema structures.

- **Environment Management:** Managing sensitive credentials via .env files and keeping a shareable .env.example improved my understanding of environment configuration best practices.

- **Troubleshooting Skills:** I resolved issues such as Docker daemon connectivity, container health checks, and authentication configuration for pgAdmin, which deepened my debugging skills.

---

4. Challenges Faced

The main challenges included:

- Configuring Docker on macOS (requiring Colima setup for virtualization).

- Ensuring that pgAdmin could connect to the PostgreSQL container using db instead of localhost.

- Understanding the correct network bridging between services defined in Docker Compose.

Through persistence and debugging logs, I was able to resolve these and achieve a stable multi-container environment.

---

5. Outcomes

By the end of this project:

- I successfully deployed and tested all three services using Docker Compose.

- Verified the FastAPI application running on localhost:8000.

- Connected pgAdmin (localhost:5050) to the PostgreSQL container.

- Executed SQL CRUD operations (CREATE, INSERT, SELECT, UPDATE, DELETE) and verified their results in pgAdmin.

- Documented each step with screenshots for evaluation.

---

6. Reflection Summary

This project strengthened my understanding of **backend engineering, containerization, and database connectivity**.
It demonstrated the value of reproducible environments in modern development workflows.
The experience also highlighted the importance of documentation, version control (via GitHub), and structured testing.
Going forward, I feel confident setting up similar full-stack data systems for production or team environments.

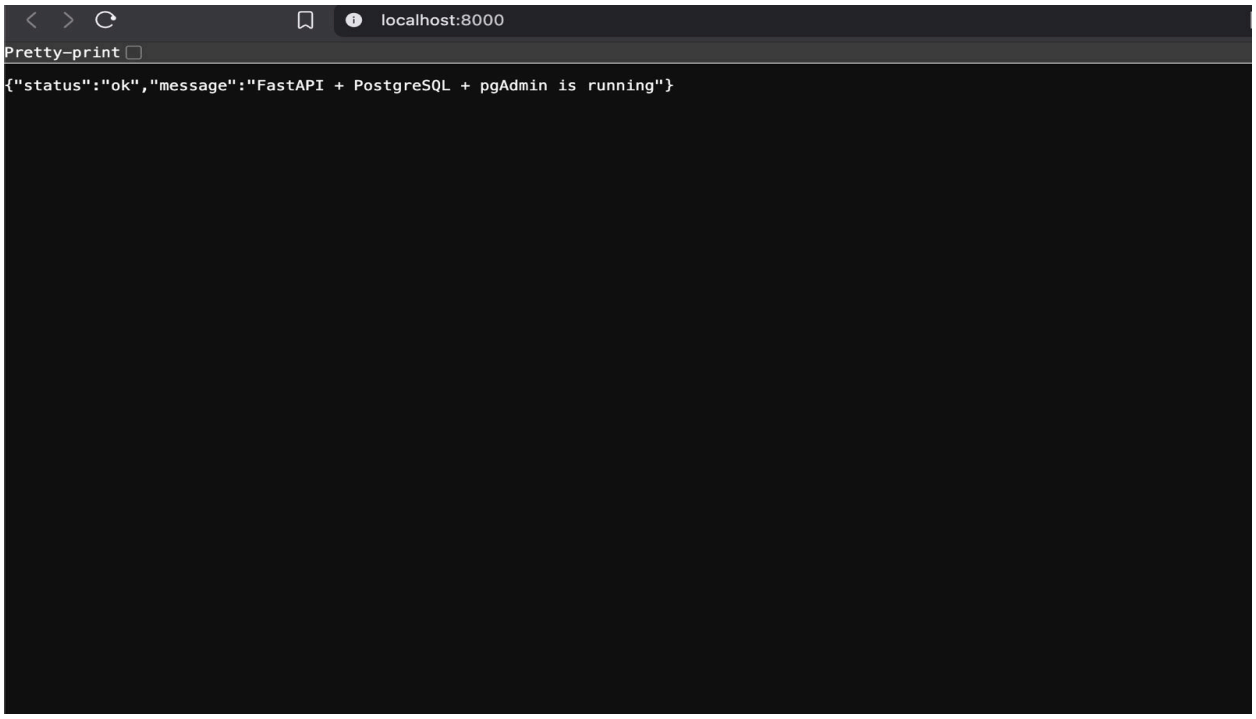
---

7. GitHub Repository

📦 **Project Link:** https://github.com/SLekhana/fastapi-calculator.

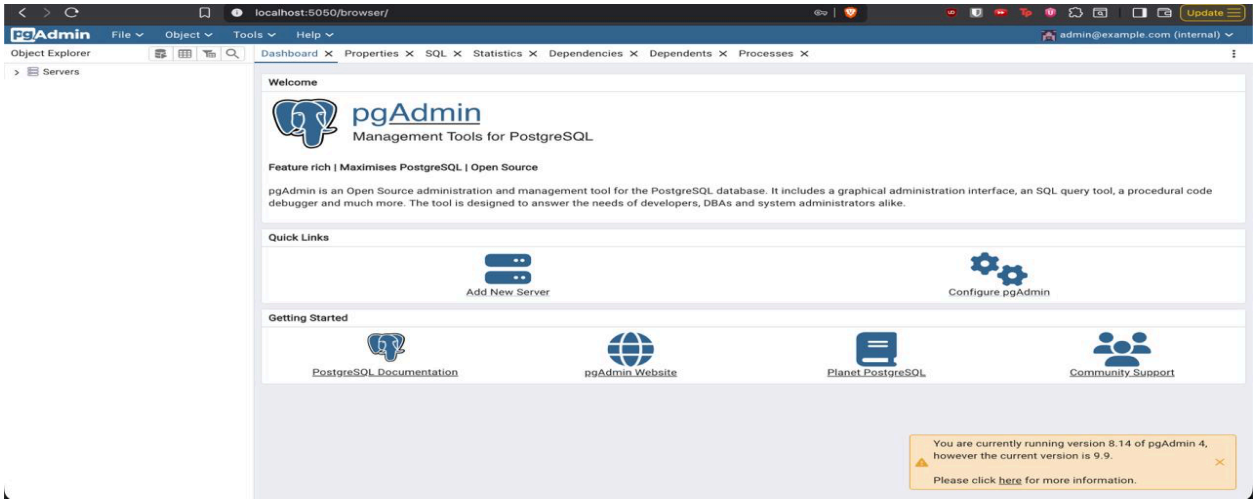
Caption:
✅ FastAPI application is running successfully on `localhost:8000`, confirming the Docker Compose setup is working.
Output shows:

{"status": "ok", "message": "FastAPI + PostgreSQL + pgAdmin is running"}



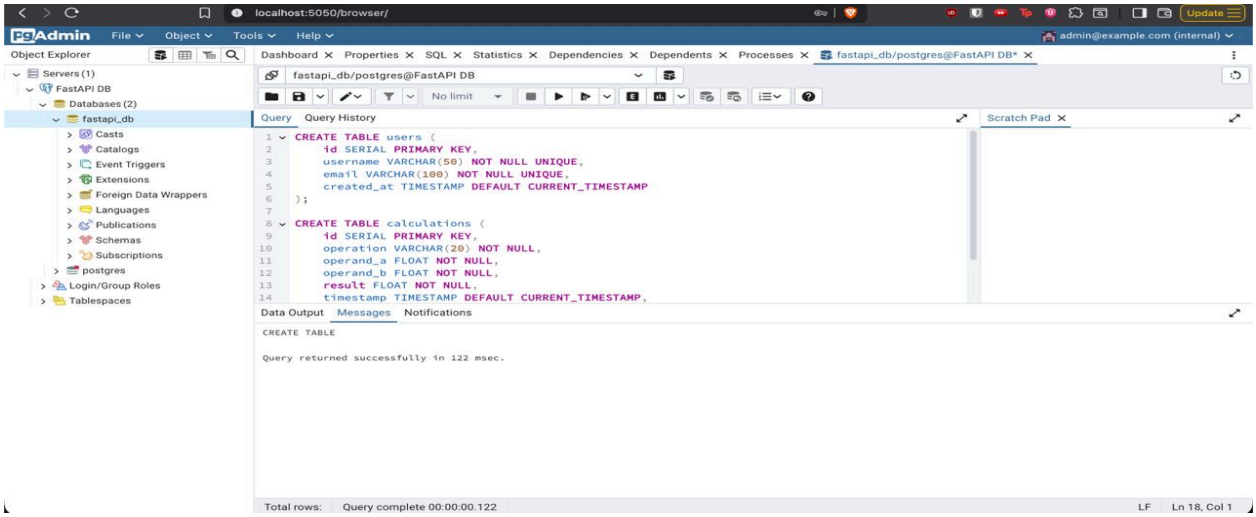
🖼️ Screenshot 2: pgAdmin Login Dashboard

Caption:
✅ pgAdmin is accessible at `localhost:5050`, confirming the web-based PostgreSQL management interface is working.
This page allows adding a new server connection to the PostgreSQL container.



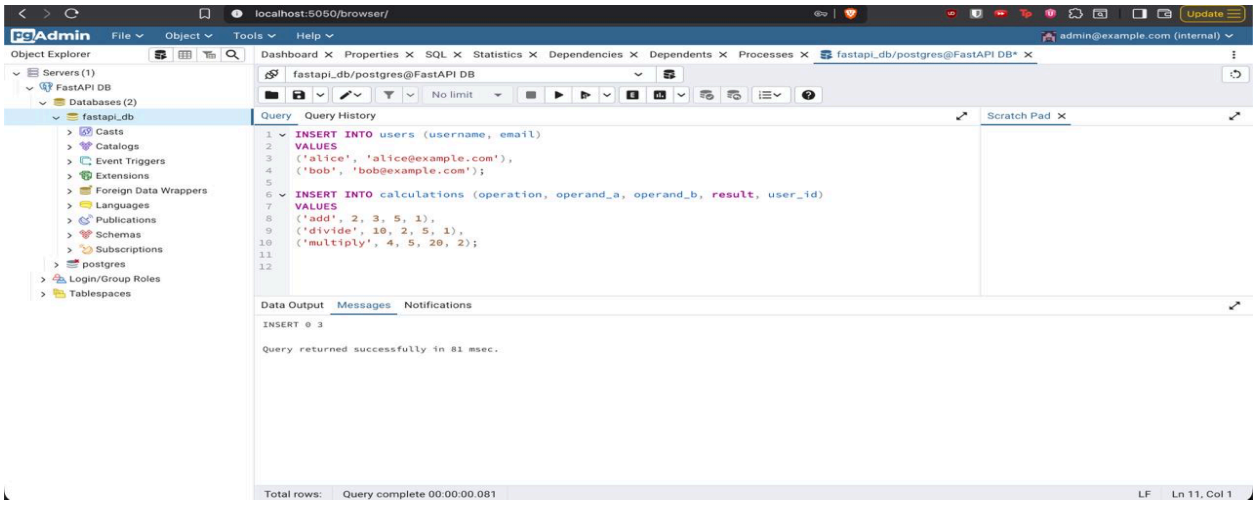
🖼️ Screenshot 3: Table Creation

Caption:
✅ SQL commands to create the `users` and `calculations` tables were executed successfully in pgAdmin.
The message *"Query returned successfully"* confirms both tables were created in the `fastapi_db` database.



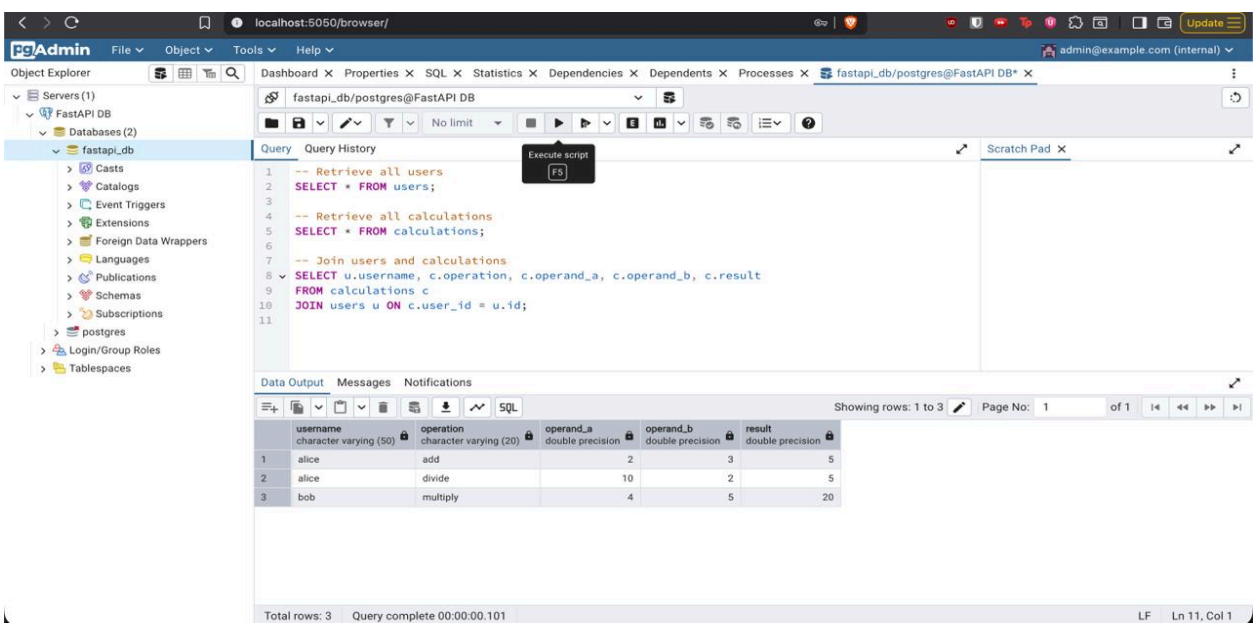
🖼️ Screenshot 4: Data Insertion

Caption:
✅ Insert statements successfully added records into the `users` and `calculations` tables.
The message *"INSERT 0 3"* indicates three calculation records were inserted.



🖼️ Screenshot 5: Data Retrieval and Join Query

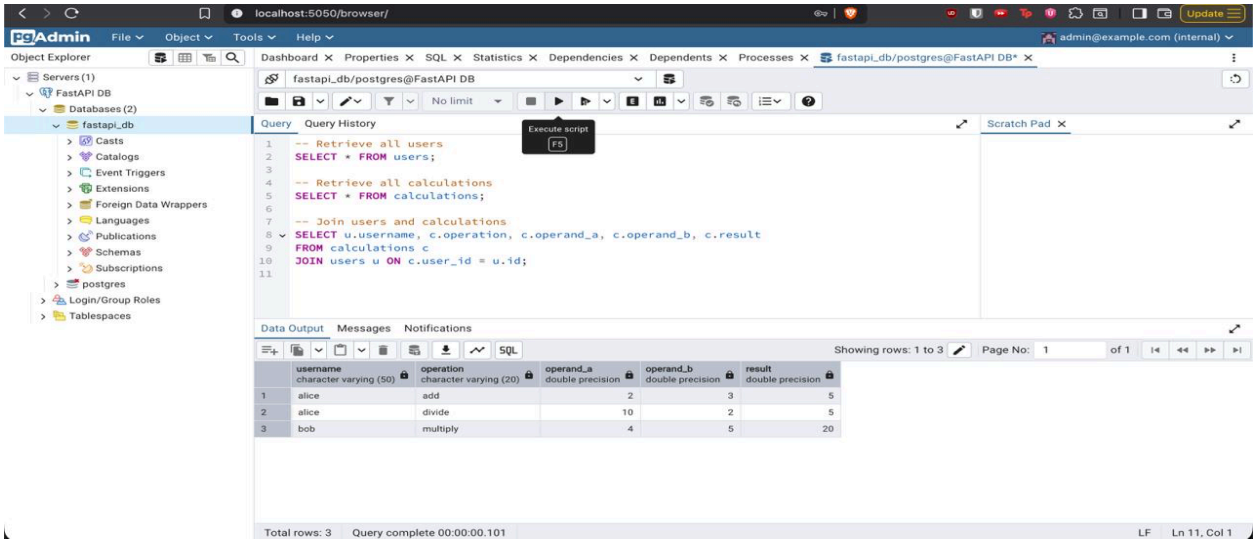
Caption:
✅ SELECT queries retrieved all users and calculations, and the JOIN query combined user and calculation data correctly.
The output shows matching usernames with their respective operations and results.



	username character varying (50)	operation character varying (20)	operand_a double precision	operand_b double precision	result double precision
1	alice	add	2	3	5
2	alice	divide	10	2	5
3	bob	multiply	4	5	20

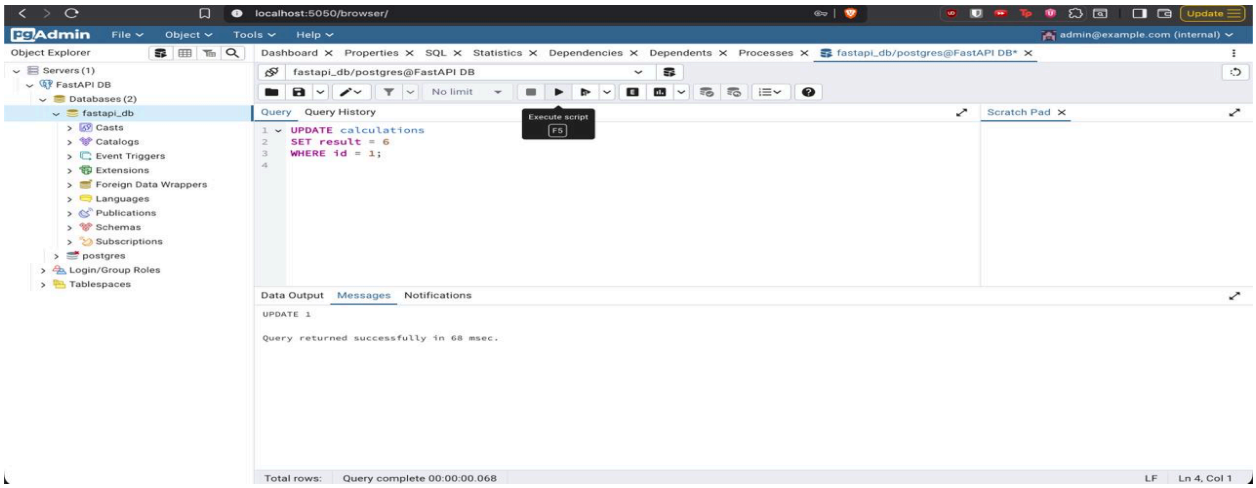
🖼️ Screenshot 5: Data Retrieval and Join Query

Caption:
✅ SELECT queries retrieved all users and calculations, and the JOIN query combined user and calculation data correctly.
The output shows matching usernames with their respective operations and results.



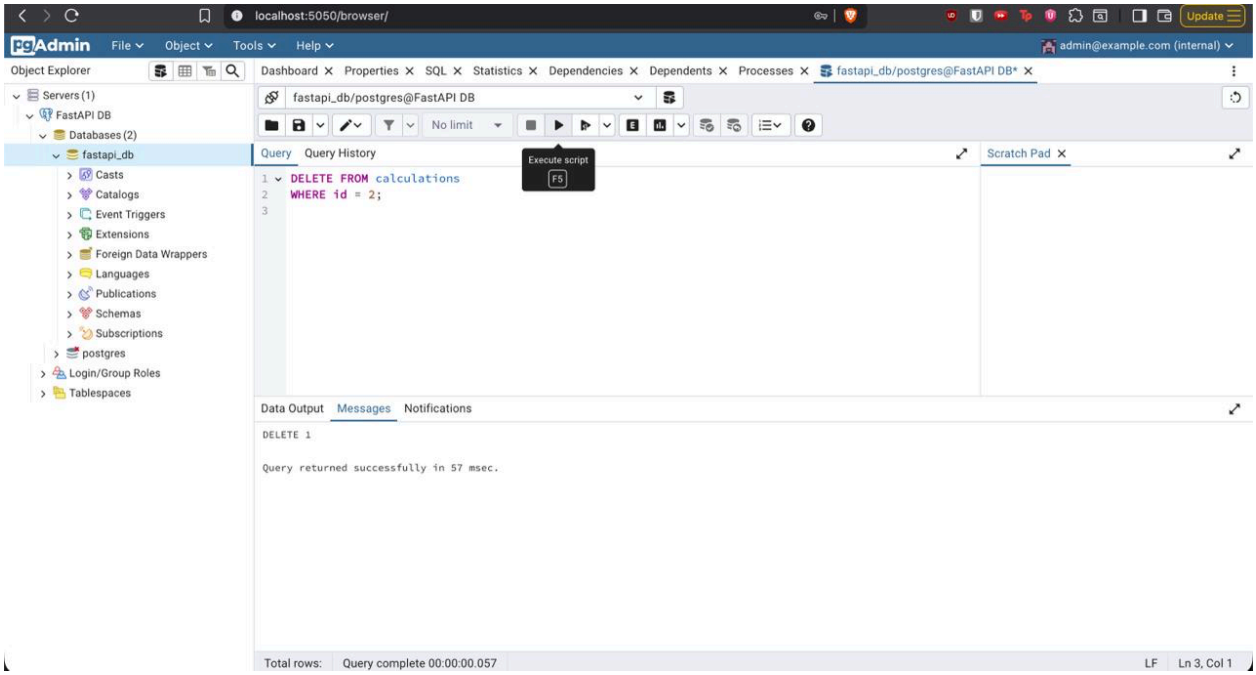
🖼️ Screenshot 6: Update Query

Caption:
✅ UPDATE operation executed successfully — the result for calculation ID 1 was changed to 6.
pgAdmin confirms with *"UPDATE 1"* message.



🖼️ Screenshot 7: Delete Query

Caption:
✅ DELETE operation executed successfully — calculation ID 2 was removed from the database.
pgAdmin confirms with *"DELETE 1"* message.



🖼️ Screenshot 8: Docker Compose Down

Caption:
✅ Docker containers (`fastapi_app`, `fastapi_db`, and `pgadmin`) were successfully stopped and removed using `docker compose down`.
This confirms proper cleanup and shutdown of all running services.